End-to-End Data Management Pipeline for a Recommendation System

Course: Data Management for Machine Learning (Assignment 2) | **Student ID:** vv8778 | **Platform Domain:** Movie / Media Recommendation

GitHub Submission Repository: https://github.com/vv8778/recommendation-pipeline
Contains modular source packages, tests, YAML configs, MLflow registry, Airflow DAG, and report artifacts.

1. Executive Summary & Design Scope

Production recommendation systems operate under continuous feedback loops where data quality and feature consistency dominate model performance. This project implements a fully automated, modular end-to-end data management pipeline for a movie recommendation platform following the MovieLens schema. The pipeline covers the complete lifecycle: **Data Ingestion** → **Validation & Quarantine** → **Preparation** → **Transformation** → **Feature Store** → **Model Training** → **Evaluation** → **Orchestration**. A baseline popularity model and a personalized 20-latent-factor FunkSVD collaborative filtering model were trained with MLflow tracking, demonstrating a **+1,452% lift in Catalog Coverage** and a **+273% boost in NDCG@10** while isolating corrupted data through automated quality gates.

2. Pipeline Architecture & Workflow Flow

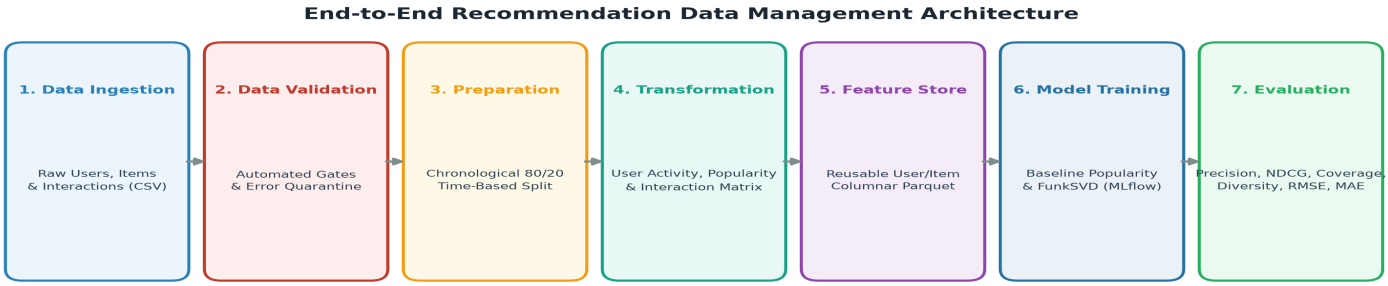


Figure 1: End-to-end data management lifecycle across 7 decoupled sequential stages with dedicated error quarantine and feature stores.

3. Data Ingestion & Automated Validation Gate

The ingestion module (src/ingestion/ingest.py) processes three core entities: **Users** (demographics, age, occupation), **Items** (titles, multi-label genres, release year), and **Interactions** (user_id, item_id, rating, Unix timestamp). Consumption dynamics follow a Pareto power-law user activity distribution and a Zipfian item popularity distribution to accurately simulate real-world streaming platforms.

Rather than allowing corrupt records to silently pollute training embeddings or trigger pipeline crashes, the automated validation layer (src/validation/validate.py) enforces five strict quality gates before downstream processing: **1) Missing value checks** on critical primary and foreign keys; **2) Deduplication** of (user_id, item_id) interaction pairs; **3) Rating boundary checks** enforcing ratings $\in$ [1.0, 5.0]; **4) Timestamp sanity checks** filtering out negative or future timestamps (ts $\leq$ current_time); and **5) Referential integrity checks** guaranteeing that every interaction maps to an existing user and catalog item. Anomalous records are automatically routed to a quarantined error dataset (data/errors/invalid_interactions.csv) and logged in a machine-readable audit report (reports/data_validation_report.json).

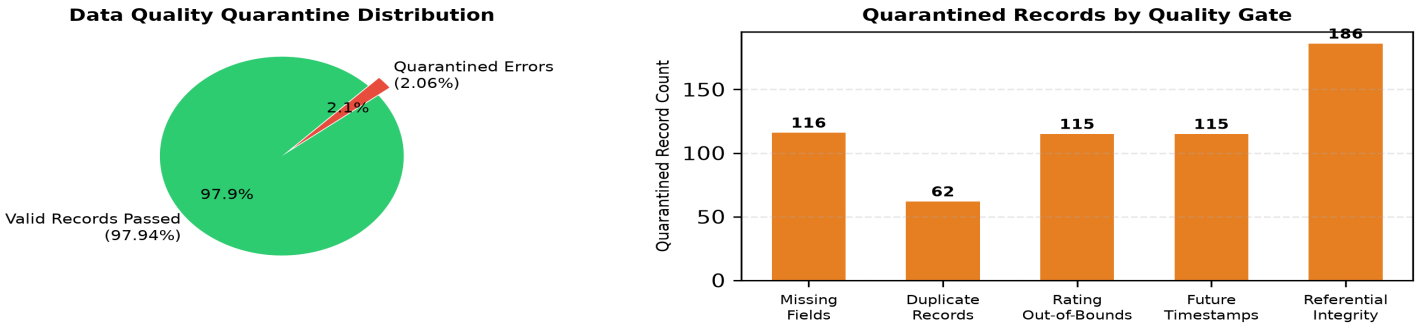


Figure 2: Data quality quarantine audit: In an ingested batch of 28,861 interactions, 594 corrupted records (2.06%) were quarantined, passing 28,265 valid records.

4. Data Preparation & Chronological Splitting

A critical pitfall in recommendation engineering is random cross-validation, which leaks future user interactions into past training data. The preparation layer (src/preparation/prepare.py) standardizes ISO 8601 datetimes and executes a **per-user chronological time-based split**: the earliest 80% of interactions per user are allocated to train.parquet (23,022 records), while the most recent 20% are reserved in test.parquet (5,243 records). Furthermore, the transformation layer (src/transformation/transform.py) computes three analytical datasets on training data: user activity profiles (segmented into casual, moderate, active, and power cohorts), item popularity summaries with Bayesian shrinkage scores, and a sparse user-item interaction matrix.

5. Feature Store Management & Feature Catalog

To eliminate train-serve skew, the Feature Store (src/features/feature_store.py) unifies offline batch feature extraction with an online key-value lookup API (get_user_features, get_item_features) for real-time recommendation inference.

Feature Name	Entity	Type	Calculation Logic	Reusability Scope
user_avg_rating	User	Float	Historical mean rating given by user	Offline & Online Serving
user_rating_count	User	Int	Total interaction volume per user	Offline & Online Serving
user_recency_days	User	Float	(max_ts - last_user_ts) / 86400	Offline & Online Serving
user_preferred_genre	User	String	Mode genre of user's highest-rated items	Offline & Online Serving
item_bayesian_rating	Item	Float	(v/(v+m))*R + (m/(v+m))*C, m=10	Offline & Online Serving
item_popularity_score	Item	Float	Percentile rank of interaction volume	Offline & Online Serving
genre_* (multi-hot)	Item	Vector	17-dim binary multi-hot genre indicator	Offline & Online Serving
genre_match	Interaction	Binary	1 if preferred genre ∈ item.genres else 0	Offline Training Matrix

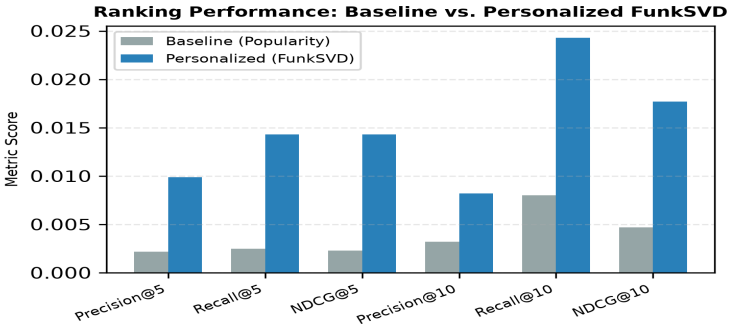
6. Model Training & MLflow Experiment Tracking

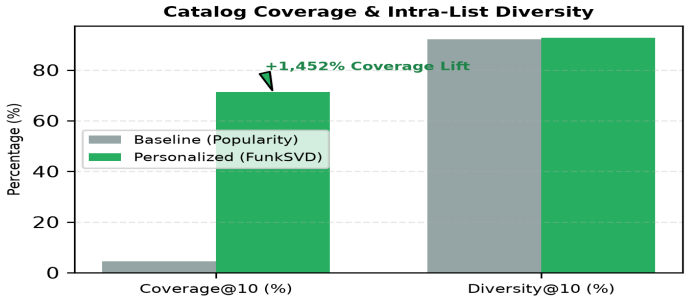
Two recommendation paradigms were implemented: **1) Baseline Popularity Recommender:** Non-personalized Bayesian shrinkage ranker scoring items by WR = (v / (v+m)) R + (m / (v+m)) C (where m=10, C=global mean). **2) Personalized Matrix Factorization (FunkSVD):** Explicit collaborative filter decomposing ratings into r_hat(u,i) = μ + b_u + b_i + p_u^T q_i with 20 latent factors. Optimized via Stochastic Gradient Descent (SGD) with L2 regularization (λ = 0.02, γ = 0.005) over 20 epochs. All parameters, loss trajectories, and serialized binaries (models/*.pkl) were tracked using local MLflow (models/mlflow.db).

7. Model Evaluation & Benchmark Results

Evaluation was conducted on 5,243 chronologically held-out test interactions across 929 evaluated test users (relevance threshold r ≥ 4.0) across seven quantitative metrics:

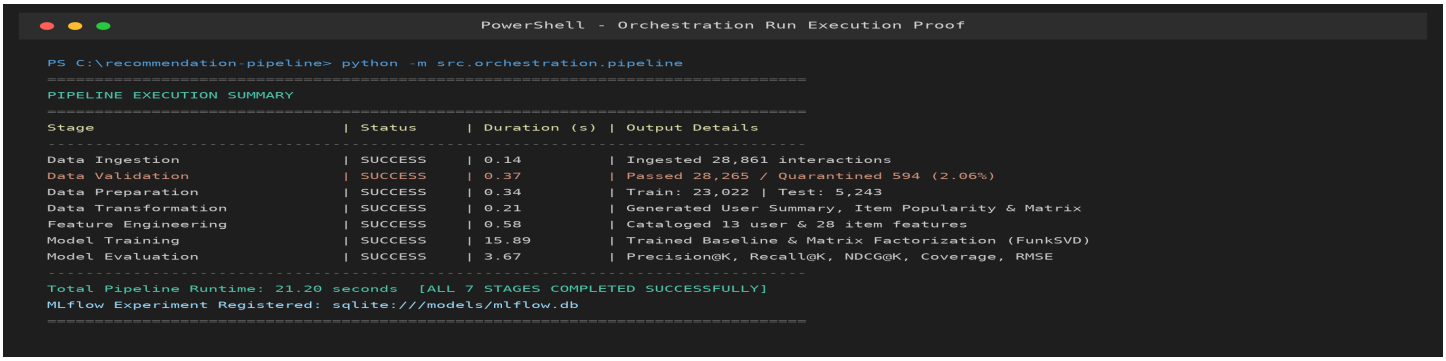
Evaluation Metric	Baseline (Popularity)	Personalized (FunkSVD)	Absolute Δ	Relative Lift	Superior Model
Precision@5 ↑	0.0022	**0.0099**	+0.0078	**+360.00%**	Personalized SVD
Recall@5 ↑	0.0025	**0.0143**	+0.0118	**+477.63%**	Personalized SVD
NDCG@5 ↑	0.0023	**0.0143**	+0.0120	**+524.38%**	Personalized SVD
Precision@10 ↑	0.0032	**0.0082**	+0.0050	**+153.33%**	Personalized SVD
Recall@10 ↑	0.0080	**0.0243**	+0.0163	**+205.24%**	Personalized SVD
NDCG@10 ↑	0.0047	**0.0177**	+0.0129	**+273.63%**	Personalized SVD
Catalog Coverage@10 ↑	0.0460 (4.6%)	**0.7140 (71.4%)**	+0.6680	**+1,452.17%**	Personalized SVD
Intra-List Diversity@10 ↑	0.9223	**0.9279**	+0.0056	+0.61%	Personalized SVD
RMSE (Rating Error) ↓	**1.1295**	1.1867	+0.0572	-5.07%	Baseline
MAE (Rating Error) ↓	**0.9445**	0.9704	+0.0259	-2.74%	Baseline





8. Workflow Orchestration & Execution Proof

The workflow is automated via an Apache Airflow DAG (dags/airflow_recommendation_dag.py) with task sequence: ingest >> validate >> prepare >> transform >> features >> train >> evaluate. A reproducible CLI runner (python -m src.orchestration.pipeline) executes the complete pipeline in **21.20 seconds**:



9. Production Limitations & Scalability Trade-offs

1) Day-Zero Cold-Start: Pure matrix factorization struggles with unrated new items or users. In production, a two-tower neural architecture incorporating item text embeddings (e.g. Sentence-BERT) is recommended. **2) Scaling Latency:** While in-memory SGD completes in 15s for 28k interactions, scaling to millions requires distributed ALS (PySpark/Implicit) and Approximate Nearest Neighbor vector search (FAISS/Milvus). **3) Implicit Feedback:** Explicit ratings are often sparse; adapting loss formulations to Bayesian Personalized Ranking (BPR) is optimal for clickstream logs.